



02. MongoDB CRUD: Create, Read & Update



Inserting Multiple Documents: insertMany()

insertMany() me documents ka array of objects pass kiya jata hai:

```
mongosh - insertMany
db.student.insertMany([
  { name: "Priya Verma", age: 19, marks: 88, city: "Mumbai" },
  { name: "Amit Patel", age: 22, marks: 75, city: "Delhi" },
  { name: "Neha Singh", age: 20, marks: 92, city: "Pune" }
]);
```



Reading Documents: find() vs findOne()

- **db.student.find()** : Matching documents ka **Cursor** (pointer) return karta hai. Filter khali ho toh saare records aate hain.
- **db.student.findOne({ name: "Amit Patel" })** : Pehla matched **single document** return karta hai, agar match na mile toh **null** return karta hai.

Cursor Methods in find():

- **.limit(2)** → Sirf pehle 2 documents return karega.
- **.sort({ marks: -1 })** → Descending order me sort karega.
- **.countDocuments()** → Total matching documents count batayega.



Updating Documents: updateOne & updateMany

Always use **\$set** operator, nahi toh pura document overwrite ho jayega!

```
mongosh - update with $set
// 1. Single document update
db.student.updateOne({ name: "Rahul Sharma" }, { $set: { marks: 98 } });

// 2. Multiple documents update
db.student.updateMany({ city: "Delhi" }, { $set: { state: "Delhi NCR" } });

// 3. replaceOne (poora document replace karta hai except _id)
db.student.replaceOne({ name: "Amit Patel" }, { name: "Amit P.", age: 23 });
```

Comparison & Logical Query Operators

Comparison Query Operators

Operator	Meaning	Code Example	Description
<code>\$eq</code>	Equal to	<code>{ age: { \$eq: 20 } }</code>	Matches values exactly equal to 20
<code>\$ne</code>	Not equal to	<code>{ city: { \$ne: "Pune" } }</code>	Matches values not equal to Pune
<code>\$gt</code>	Greater than	<code>{ marks: { \$gt: 85 } }</code>	Marks greater than 85
<code>\$gte</code>	Greater or equal	<code>{ marks: { \$gte: 90 } }</code>	Marks 90 ya usse zyada
<code>\$lt</code>	Less than	<code>{ age: { \$lt: 21 } }</code>	Age 21 se kam
<code>\$lte</code>	Less or equal	<code>{ age: { \$lte: 20 } }</code>	Age 20 ya usse kam
<code>\$in</code>	In array	<code>{ city: { \$in: ["Delhi", "Mumbai"] } }</code>	Value array ke kisi element se match kare
<code>\$nin</code>	Not in array	<code>{ city: { \$nin: ["Pune"] } }</code>	Value array ke kisi element me na ho

Logical Query Operators (`$or`, `$and`, `$not`, `$nor`)

```

mongosh - Logical Operators

// 1. $or: Kisi ek condition ka true hona kaafi hai
db.student.find({
  $or: [
    { marks: { $gt: 90 } },
    { city: "Delhi" }
  ]
});

// 2. $and: Dono conditions ka true hona zaroori hai
db.student.find({
  $and: [
    { age: { $gte: 20 } },
    { marks: { $gte: 80 } }
  ]
});
  
```

CASE SENSITIVITY TRAP: MongoDB me string values strictly case-sensitive hoti hain! "Delhi" aur "delhi" alag hain. Case-insensitive search ke liye Regex use karein: `{ city: /^delhi$/i }`.